

Examen de la segunda convocatoria (13/09/2022)

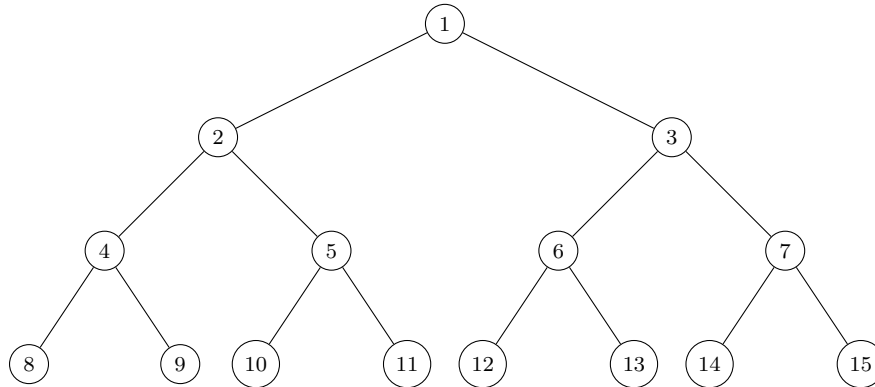
- Leer completamente cada ejercicio antes de empezar a trabajar.
- Resolver los ejercicios en **hojas separadas**.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería. Hacer un procedimiento o función que resuelve el problema **no** es resolver el ejercicio: se trata de **incidir en los aspectos de especificación y diseño**.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: “como no se indica lo contrario, supongo que ...”).
- **Comentar** y **anotar** los programas adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Todas las hojas deben llevar apellidos, nombre y número de orden.
- Duración máxima del examen: **3 horas**

Ej. 1 — (2,5 puntos) Para sensibilizar los niños sobre la protección de los animales, en un colegio se ha decidido que cada niño apadrine un animal mediante una adopción simbólica de una especie diferente. Hay n niños en el colegio: cada niño ha dado su preferencia para cada una de las n especies diferentes que se pretenden apadrinar (es decir, 0 no me gusta, 1 me gusta). Diseña un algoritmo de ramificación y poda que determine la asignación niño-animal que maximice la satisfacción global. Esta se calcula sumando las preferencias dadas por los niños por apadrinar una especie. Para ello, considera el esquema de ramificación y poda de **mínimo coste** (tal como se ha visto en los ejemplos en clase) y define:

- a)** Una representación para las soluciones, precisando las restricciones explícitas e implícitas, y el árbol del espacio de soluciones.
- b)** La función de coste $c(x)$, para cada nodo x del árbol, que hay que minimizar.
- c)** Una función de estimación adecuada, $\hat{c}(x)$, para ser utilizada como función de prioridad de los nodos vivos.
- d)** Una función de poda adecuada, $\mathcal{U}(x)$.

Ej. 2 — (2,5 puntos) Como si fuera una máquina de pinball se dejan caer una cantidad de n bolas, una por una, desde la raíz de una estructura de árbol binario completo. Todos los nodos del árbol están numerados de forma secuencial, comenzando en 1 con la raíz (nivel 1), y luego los hijos de nivel 2, y así sucesivamente. Los nodos en cualquier nivel están numerados de izquierda a derecha. Cada vez que una bola se deja caer, visita primero un nodo no terminal del árbol, continúa moviéndose hacia abajo, ya sea siguiendo el camino del sub-árbol izquierdo o el camino del sub-árbol derecho, hasta que llega en uno de los nodos hoja del árbol. Para determinar la dirección de movimiento de una bola, cada nodo no terminal tiene asociado un valor booleano (*flag*): falso o verdadero. Al comienzo del juego, todos los *flag* tienen valor falso. Al visitar un nodo no terminal, si el valor actual del *flag* en este nodo es falso, la bola primero cambiará el

valor del nodo, es decir, de falso a verdadero, y luego seguirá el sub-árbol izquierdo de este nodo para seguir moviéndose hacia abajo. De lo contrario, también cambiará el valor del *flag*, es decir, de verdadero a falso, pero seguirá el sub-árbol derecho de este nodo para seguir moviéndose hacia abajo. Por ejemplo, la figura a continuación representa un árbol binario completo de nivel 4.



Puesto que todos los *flag* están configurados inicialmente como falsos, la primera bola que se deja caer cambiará los valores de los *flag* en los nodos 1, 2 y 4 antes de que finalmente se detenga en la posición 8. La segunda bola que se suelta cambiará los valores de los *flag* en los nodos 1, 3 y 6, y se detendrá en la posición 12. La tercera bola que se deja caer cambiará los valores de los *flag* en los nodos 1, 2 y 5 antes de que se detenga en posición 10.

Diseña un algoritmo *divide y vencerás* que tome en entrada: 1) la profundidad máxima del árbol P y 2) la i -ésima bola, y determine la posición de parada de la bola. Puedes asumir que el valor i es menor o igual al número total de hojas del árbol.

Ej. 3 — (2,5 puntos) El profesor Sopmac quiere decorar el pasillo de su casa con cuadros. Como supone que sus visitantes se fijarán mejor en ellos si se encuentran próximos a las puertas, ha marcado en una recta las posiciones de las mismas.

Calcula que un cuadro está bien colocado si se encuentra a una distancia menor o igual de 0,75 metros de una puerta, y además piensa que cada una debería tener cerca un cuadro.

Lo que quiere es saber dónde colocar los cuadros de manera que se minimice el número que necesita. Para ello, debemos diseñar un algoritmo que encuentre una solución óptima.

Ej. 4 — (2,5 puntos) Dados N objetos de longitud $l_1, l_2, \dots, l_N$, (naturales positivos) y un número natural L , se desea saber si existe algún subconjunto de los N objetos cuya suma de longitudes sea exactamente L .

1. Diseñar una ecuación en recurrencias que devuelva verdad si existe tal subconjunto y falso en caso contrario.
2. Diseñar un algoritmo iterativo para la ecuación anterior con un coste en espacio de $O(L)$. Calcular el coste en tiempo.

Indicar cómo se modificaría el algoritmo anterior para que adicionalmente se obtenga uno de esos subconjuntos de objetos (en caso de que exista alguno). Calcular el coste en tiempo y en espacio de la solución.